

Project #3: Single-Cycle CPU

Dongmoon Min
(Sungkyunkwan University)
dongmoon.min@skku.edu

Projects of This Semester

	What we will make
Project #1	010-Detector
Project #2	ALU & Register File
Project #3	Single-Cycle CPU

- ◆ Language: Verilog
- ◆ Framework: AMD Vivado, Icarus Verilog

Team Announcement

- ◆ Two to three students become one team.
- ◆ You can find your teams in “사용자 및 그룹” → “그룹”



컴퓨터구조개론_SWE3005_42(민동문) > 사용자 > 그룹

2025년 1학기

전체 **그룹**

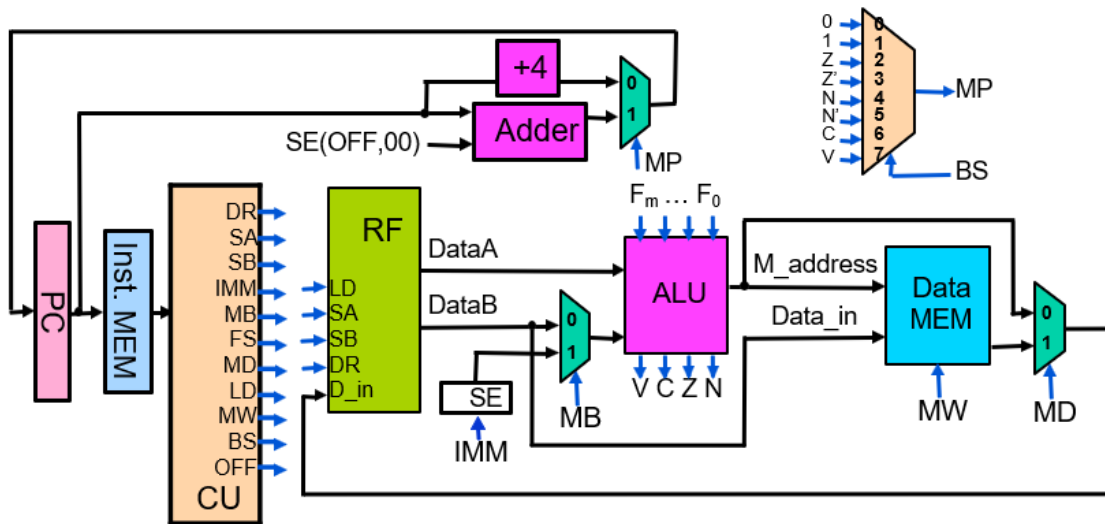
그룹 또는 사용자 검색

▶ 프로젝트 1 프로젝트	학생 2 명	🔒
▶ 프로젝트 2 프로젝트	학생 2 명	🔒
▶ 프로젝트 3 프로젝트	학생 3 명	🔒
▶ 프로젝트 4 프로젝트	학생 2 명	🔒
▶ 프로젝트 5 프로젝트	학생 2 명	🔒

Single-Cycle CPU

Goals

- ◆ Understand a CPU architecture
- ◆ Implement a single-cycle CPU using Verilog

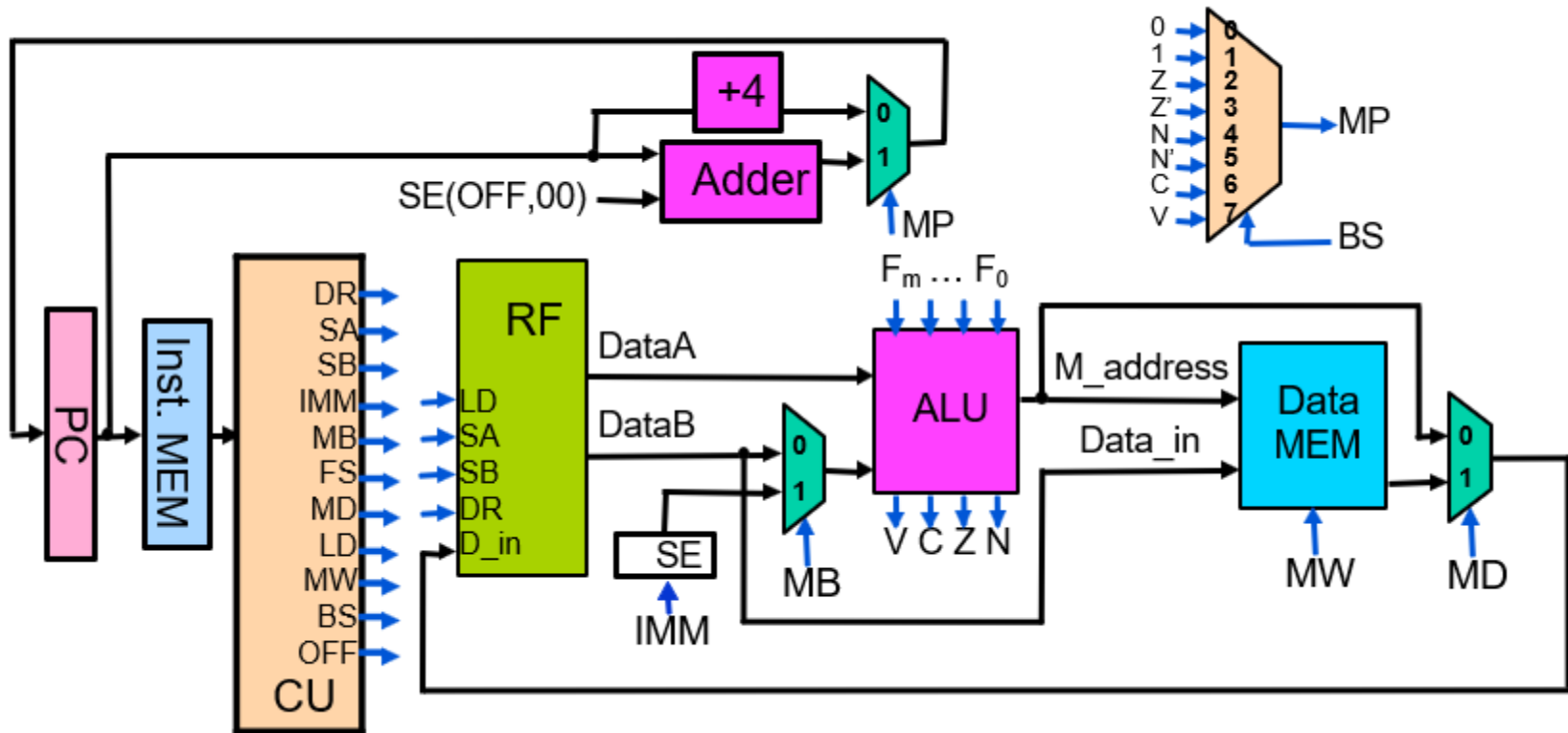


← Implement this slide!

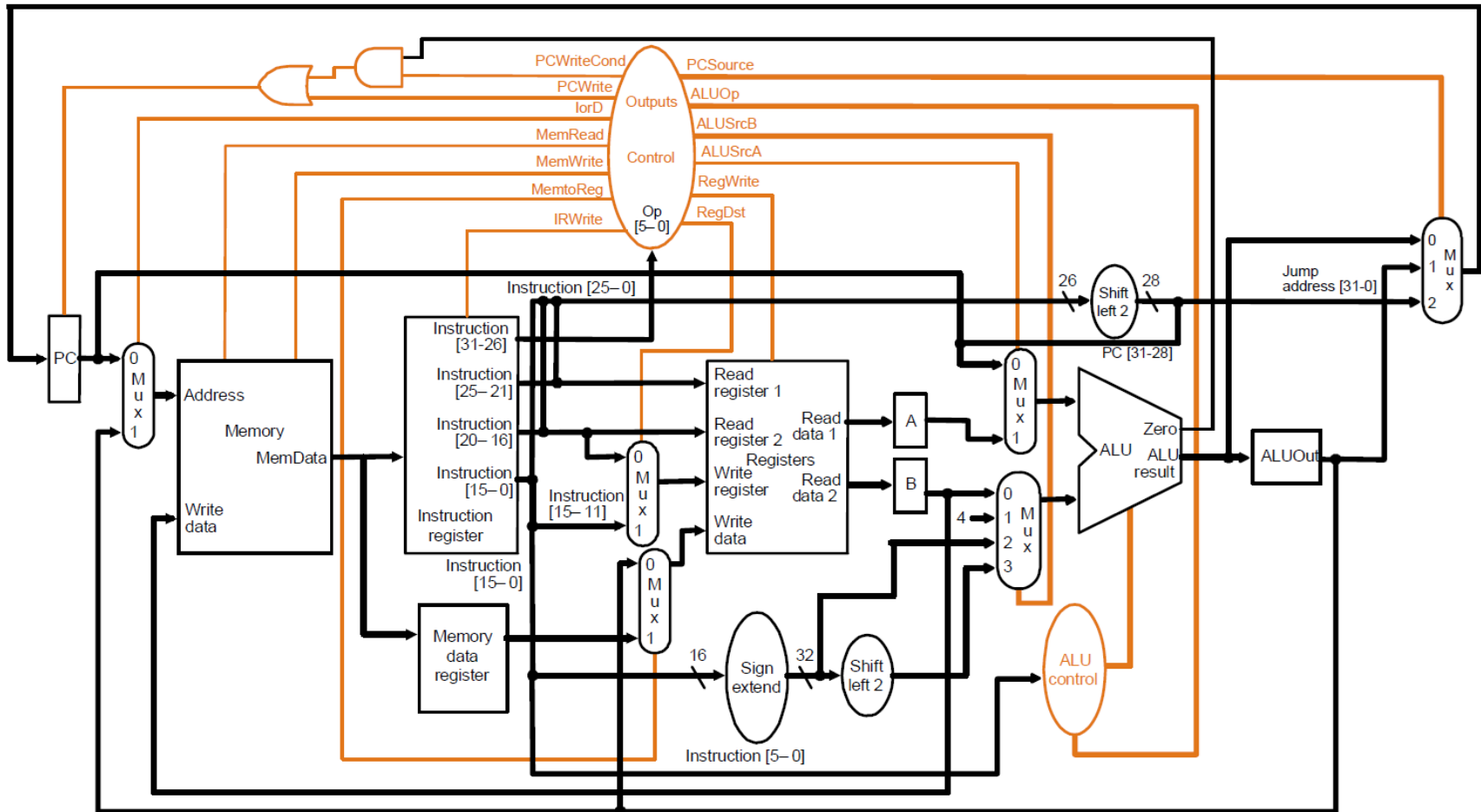
Project Assignment

- ◆ Implement a single-cycle TSC CPU.
 - In this assignment, you will implement a part of the TSC ISA
 - **ADD, ADI, LHI, JMP, WWD**
 - Each instruction should be executed in a single cycle.
- ◆ The lecture note will be the best reference design.
- ◆ **But, you can do whatever you want, as long as it works as a single-cycle CPU.**

Reference Single-Cycle CPU Designs



Reference Single-Cycle CPU Designs



Important Announcement

- ◆ We will provide a CPU skeleton code.
- ◆ For the port description, please refer to the comments
- ◆ **Do not touch input and outputs ports inside skeleton code and testbench!!!**

```

1  ///////////////////////////////////////////////////////////////////
2  // MODULE: CPU for TSC microcomputer: cpu.v
3  // Author:
4  // Description:
5
6  // DEFINITIONS
7  `define WORD_SIZE 16    // data and address word size
8
9  // MODULE DECLARATION
10 module cpu (
11     output readM,          // read from memory
12     output [`WORD_SIZE-1:0] address, // current address for data
13     inout [`WORD_SIZE-1:0] data,    // data being input or output
14     input inputReady,          // indicates that data is ready from the input port
15     input reset_n,            // active-low RESET signal
16     input clk,                // clock signal
17
18     // for debugging/testing purpose
19     output [`WORD_SIZE-1:0] num_inst, // number of instruction during execution
20     output [`WORD_SIZE-1:0] output_port // this will be used for a "WWD" instruction
21 );
22
23
24
25
26
27
28 endmodule
    
```

Do not touch here!

ISA

- ◆ Instruction Set Architecture
 - User manual of the computer
 - Programmer visible state

- ◆ We will use TSC ISA instead of MIPS ISA
 - 16-bit RISC ISA
 - Similar to MIPS, but much more simplified.
 - Refer to the manual on i-Campus.

◆ Instruction types

- | | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|----------|--------|----|----|----|----------------|----|----|---|--------------------|---|----------|---|---|---|---|---|
| R-format | opcode | | | | rs | | rt | | rd | | function | | | | | |
| I-format | opcode | | | | rs | | rt | | immediate / offset | | | | | | | |
| J-format | opcode | | | | target address | | | | | | | | | | | |

TSC ISA (2/2)

- ◆ Instruction Opcodes
 - Please refer to the manual!
 - tsc_assem.pdf

Instruction	Opcode	Function Code	Format
[ADD]	15	0	R
[SUB]	15	1	R
[AND]	15	2	R
[ORR]	15	3	R
[NOT]	15	4	R
[TCP]	15	5	R
[SHL]	15	6	R
[SHR]	15	7	R
[ADI]	4	-	I
[ORI]	5	-	I
[LHI]	6	-	I
[RWD]	15	27	R
[WWD]	15	28	R
[LWD]	7	-	I
[SWD]	8	-	I

Tips: Implementation

```
18 // for debugging/testing purpose
19 output [`WORD_SIZE-1:0] num_inst, // number of instruction during execution
20 output [`WORD_SIZE-1:0] output_port // this will be used for a "WWD" instruction
```

◆ Output **num_inst**

- The number of instructions during execution
- In case of single-cycle CPU, you should increase it by one for every posedge of clock signals.

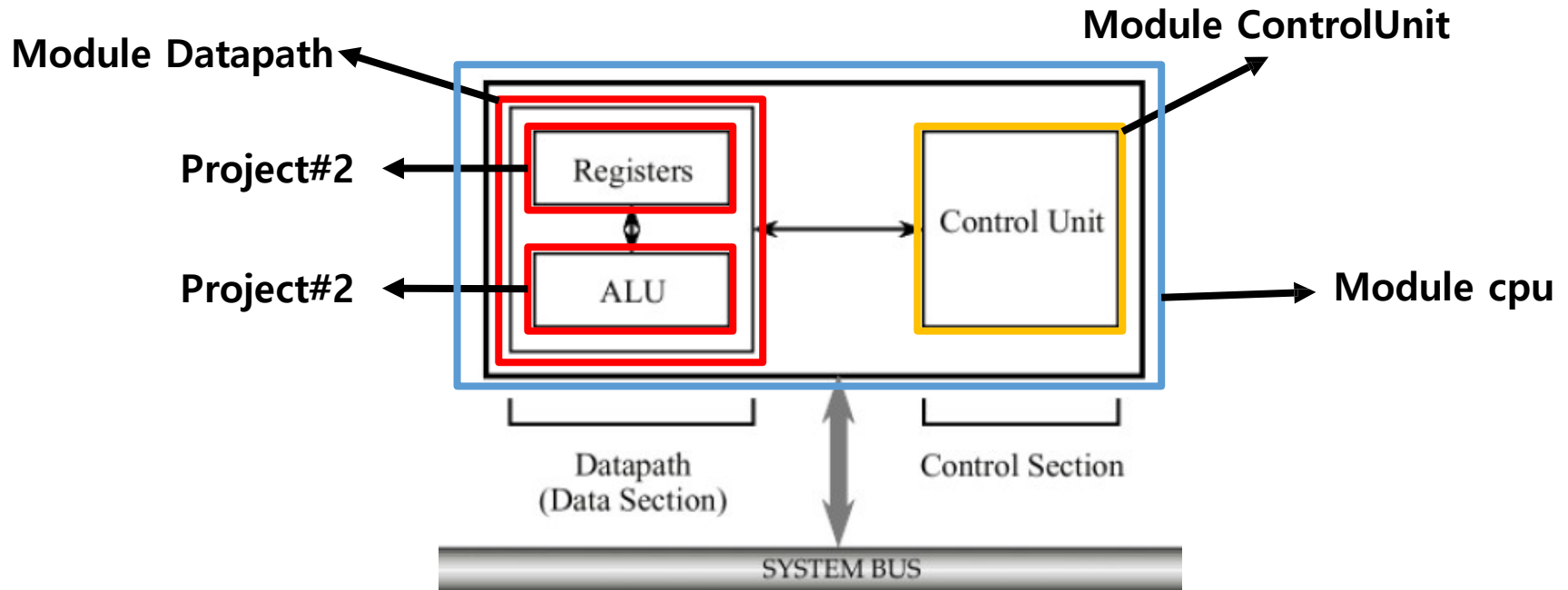
◆ Output **output_port**

- Used only for WWD instructions
- WWD instructions ([WWD \$rs]) returns \$rs to output port.

◆ Testbench checks **num_inst** and **output_port**

- Please refer to the testbench file.

Tips: Implementation



- ◆ Make your own datapath and Control Unit
 - The datapath should contain an ALU and register file (Project #2)
 - The cpu module may include a Datapath and a Control Unit module.

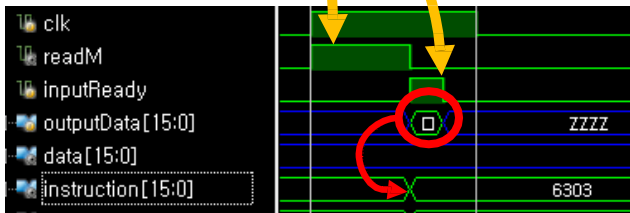
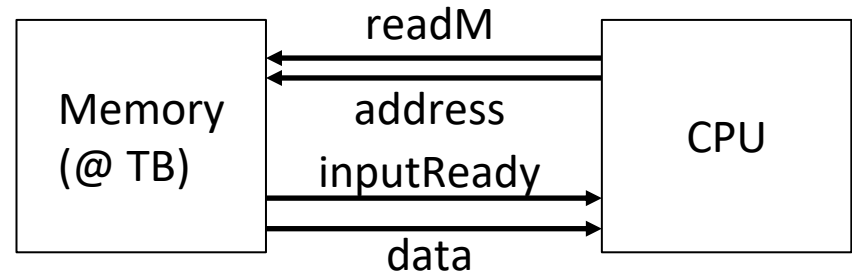
Tips: Testbench

```
53 // model the read process for the memory device
54 assign data = readM ? outputData : `WORD_SIZE'bz;
55 always begin
56     outputData = `WORD_SIZE'bz;
57     #`PERIOD1;
58     forever begin
59         wait (readM == 1);
60         #`READ_DELAY;
61         outputData = memory[address];
62         inputReady = 1;
63         #(`STABLE_TIME);
64         outputData = `WORD_SIZE'bz;
65         inputReady = 0;
66     end // of forever loop
67 end // of always block for memory read
```

- ◆ We also provide a testbench for CPU module (cpu_tb.v)
 - It contains memory code and instructions for test cases

data == *outputData* only when *readM* is 1.

```
always begin
    outputData = `WORD_SIZE'bz;
    #`PERIOD01;
    forever begin
        wait (readM == 1);
        #`READ_DELAY;
        outputData = memory[address];
        inputReady = 1;
        //$display("readM: %d, data: %d", outputData, data);
        #(`STABLE_TIME);
        outputData = `WORD_SIZE'bz;
        inputReady = 0;
    end // of forever loop
end // of always block for memory read
```

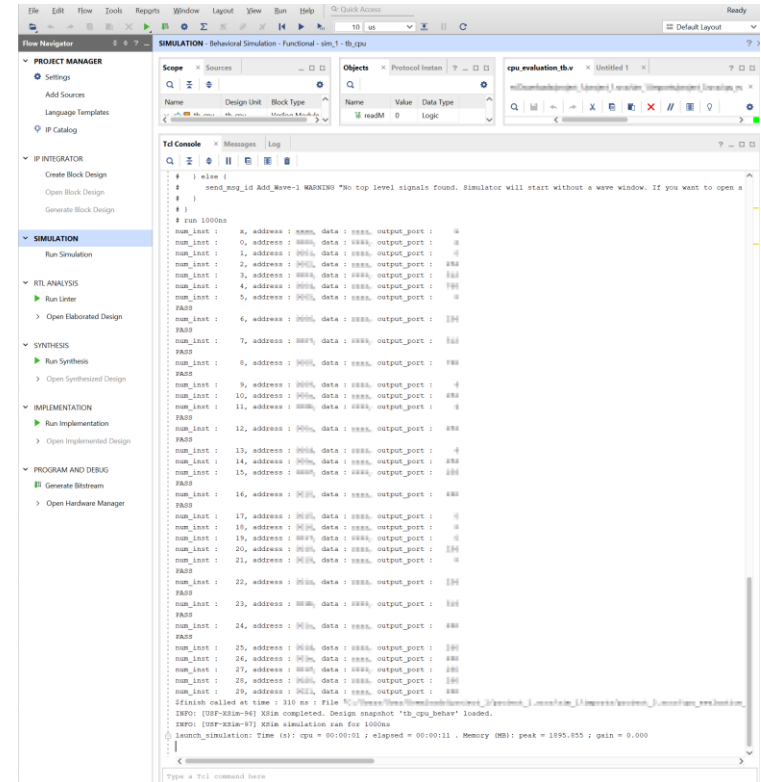


- ◆ Memory provide an instruction when readM is 1
 - While turning on inputReady as 1

Testbench: Instruction

```

69 // store programs and data in the memory
70 initial begin
71     #`PERIOD1; // delay for a while
72     memory[0] = 16'h6000; // LHI $0, 0
73     memory[1] = 16'h6101; // LHI $1, 1
74     memory[2] = 16'h6202; // LHI $2, 2
75     memory[3] = 16'h6303; // LHI $3, 3
76     memory[4] = 16'hf01c; // WWD $0
77     memory[5] = 16'hf41c; // WWD $1
78     memory[6] = 16'hf81c; // WWD $2
79     memory[7] = 16'hfc1c; // WWD $3
80     memory[8] = 16'h4204; // ADI $2, $0, 4
81     memory[9] = 16'h47fc; // ADI $3, $1, -4
82     memory[10] = 16'hf81c; // WWD $2
83     memory[11] = 16'hfc1c; // WWD $3
84     memory[12] = 16'hf6c0; // ADD $3, $1, $2
85     memory[13] = 16'hf180; // ADD $2, $0, $1
86     memory[14] = 16'hf81c; // WWD $2
87     memory[15] = 16'hfc1c; // WWD $3
88     memory[16] = 16'h9015; // JMP 21
89     memory[17] = 16'hf01c; // WWD $0
90     memory[18] = 16'hf180; // ADD $2, $0, $1
91     memory[19] = 16'hf180; // ADD $2, $0, $1
92     memory[20] = 16'hf180; // ADD $2, $0, $1
93     memory[21] = 16'h6000; // LHI $0, 0
94     memory[22] = 16'h4000; // ADI $0, $0, 0
95     memory[23] = 16'hfd80; // ADD $2, $3, $1
96     memory[24] = 16'hf01c; // WWD $0
97     memory[25] = 16'hf41c; // WWD $1
98     memory[26] = 16'hf81c; // WWD $2
99     memory[27] = 16'hfc1c; // WWD $3
    
```



- ◆ We also provide CPU testcase code in Testbench (cpu_tb.v)
 - TSC instructions in memory
 - You can check the result on Tcl console.
 - If you pass the test-case, then "PASS" is printed on the console.

Tips: Coding Style

- ◆ Please follow the coding guidelines in your project
 - That we learned in Lecture 7
 - Blocking for combinational logic
 - Nonblocking for sequential logic & memory
 - Don't mix blocking & nonblocking in the same always block
 - Don't try to change same variable in different always blocks

- ◆ If your code doesn't follow the design guidelines, your project will be highly likely to encounter catastrophic timing errors!!!

Submission

- ◆ Deadline: Refer to the announcement
- ◆ Submit your **Verilog codes only** (i.e., cpu.v, cpu_tb.v) by compressing them to zip file (*.zip).
 - File name: proj3_2025XXXXXX_김아무개_2025XXXXXX_이아무개.zip
- ◆ You should change only "cpu.v files"
 - If you change others (i.e., module name, input & output of cpu, testbench), we will give you zero score for your project!
- ◆ If you do not follow submission format (e.g., name of zip file), we will deduce 10% from your project score.

Thank You
&
Good Luck :)